Page No.
Date : / /

Stochastic Gradient Descent:

Batch watch all row then update

but stochastic do update after a row

Hence → faster conversion
→ so low RAM occupies

- Here, row will be selected randomly.

Disadvantage: - not totally precise
- may give diff answer if we run multiple times.

```
self.intercepts = 0
self.coef_ = np.ones( X_train.shape[1])

for i in range(self.epochs):
    for j in range (X_train.shape[0]):
        idx = np.random.randint(0, X_train.shape[0]);
        y_hat = np.dot(x_train[idx], self.coef_)
                + self.intercept_
        intercept_der = -2 * ~~np.mean(y_train - y_hat)~~
                (y_train[idx] - y_hat).
        self.intercept_ = self.intercept_ -
                (self.lr * intercept_der)
```

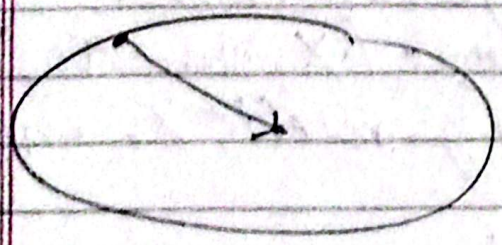
Page No.
Date: / /

```
coef_der = -2 * np.dot((y_train[idx] - y_hat),
                X_train[idx])
self.coef_ = self.coef_ - (self.lr * coef_der)
```

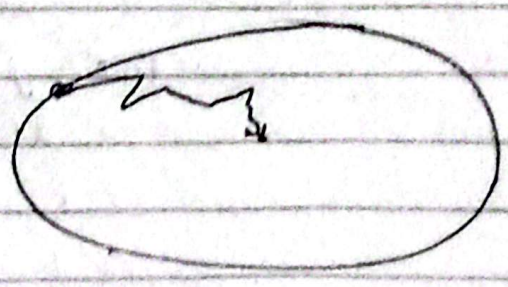
we need less epoches for Stochastic Gradient Descent. than batch GD. So, it's faster. It takes low RAM so also faster

But for equal epoches Batch ~~& Stochastic~~ will be faster than Stochastic GD

Batch.



Stochastic

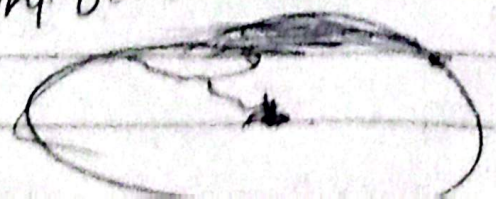


When to use Stochasic GD?

→ big data
→ Non-convex function.

{ we can vary learning rate by using fn }

mini batch.



Page No.
Date: / /

Learning Schedules.

Stochastic GD doesn't remain in same place if after getting the answer. It flutuate. To solve this learning schedules is used.

In learning schedules,
we ~~fluctuate~~ changes the learning rate.

```
t0, t1 = 5, 50.
def learning_rate(t):
    return t0/(t+t1)

for i in range(epochs):
    for j in range(X.shape[0]):
        lr = learning_rate(i*X.shape[0]+j)
```

0.1 ⟶ 0.03.

SGD stabilize when η will be low.

Types of learning rate

- Constant
- optimal
- invscaling
- adaptive

} we are not learning now.

Page No.
Date: / /

max_iter : → epoch
tol : [1e-3]
↳ tolerate if the change in ans be less than [1e-3] then it stops automatically.

do scaling then,

```
from sklearn.linear_model import SGDRegressor
reg = SGDRegressor( max_iter= 100, learning_rate
                    ='constant', eta = 0.01)
reg.fit (X_train, y_trai)
y_pred = reg.predict (X_test).
r2_score ( y_pred, y_test)
```

Page No.
Date : / /

Mini Batch GD:

1000 rows → divides in 100 ~~group~~ rows

10 groups of 100 rows.

Use loop to divide.

```
for j in range(int(X_train.shape[0]/self.batch_size));
```

Sklearn

```
import random
from sklearn.linear_model import SGDRegressor
sgd = SGDRegressor(learning_rate='constant', eta=0.2)
batch_size = 35
for i in range(100):
    idx = random.sample(range(X_train.shape[0]),
            batch_size)
    sgd.partial_fit(X_train[idx], y_train[idx])
    # partial_fit does epoch=1, max_iter=1.
```